

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

**Отчет о программном проекте на тему:
Алгоритмы предсказания цены и оптимизации**

(промежуточный, этап 1)

Выполнили студенты:

группы #БПМИ234, 3 курса
группы #БПМИ234, 3 курса
группы #БПМИ2310, 3 курса

Шадрин Андрей Романович
Климов Демьян Дмитриевич
Барсуков Борис Михайлович

Принял руководитель проекта:

Мунерман Илья Викторович
Руководитель Интерфакс-ЛАБ, к.э.н.

Москва 2026

Содержание

Аннотация	3
Ключевые слова	3
1 Описание используемых алгоритмов	4
1.1 Обнаружение точек смены режима	4
1.2 Рекуррентные нейронные сети (RNN, LSTM, GRU)	4
1.3 Свёрточные нейронные сети для временных рядов	7
1.4 Модели на основе механизма внимания (Transformer)	7
1.5 Формирование портфеля и оптимизация по риску	8
1.6 Метрики качества прогноза и эффективности стратегии	9
1.7 Существующие модели предсказания прменных рядов, основные на многоагентной симмуляции	9
1.8 Инструменты анализа структуры временных рядов	10
2 Модели прогнозирования доходности	11
2.1 Подбор гиперпараметров	11
2.2 ARIMA	11
2.2.1 Архитектура	11
2.3 Реализация	11
2.4 LightGBM	12
2.4.1 Архитектура	12
2.4.2 Реализация	13
2.5 LSTM	14
2.5.1 Архитектура	14
2.5.2 Реализация	15
2.6 GRU	16
2.6.1 Архитектура	16
2.6.2 Реализация	16
2.7 CNN	17
2.7.1 Архитектура	17
2.7.2 Реализация	17
2.8 Transformer	19
2.8.1 Архитектура	19
2.8.2 Реализация	19
2.9 Метрики оценки	20
3 Архитектура пайплайна эмбедингов и кластеризации	22
3.1 Выбор данных	22
3.2 Обучение сети	23
3.3 Работа с данными	24
3.4 Данные	24
3.5 Предобработка данных и создание признаков	25
3.6 Методы построения эмбендингов	26
3.7 Handmade Embedder	27
Список литературы	27

Аннотация

В работе рассматривается задача прогнозирования цен акций на Московской бирже и формирования на этой основе инвестиционного портфеля. Планируется построить модель предсказания цен, использовать её для отбора бумаг и расчёта весов портфеля и в конечном итоге реализовать торгового агента, способного принимать решения о сделках на бирже с целью получения прибыли. Эффективность подхода будет проверяться на исторических данных и при бэктестировании. Практический результат — прототип системы прогнозирования, формирования портфеля и автоматизированной торговли на МОЕХ с оценкой потенциальной доходности и рисков.

Ключевые слова

прогнозирование цен, Московская биржа, формирование портфеля, торговый агент, алгоритмическая торговля, бэктестирование, машинное обучение

1 Описание используемых алгоритмов

В главе приведено описание алгоритмов и методов, которые планируется использовать в проекте: предобработка временных рядов с обнаружением точек смены режима, модели прогнозирования (RNN, LSTM, GRU, CNN, Transformer), формирование портфеля и метрики оценки. Все перечисленные методы будут задействованы на этапах плана работы.

1.1 Обнаружение точек смены режима

При работе с рыночными данными Московской биржи важно выявлять структурные сдвиги во временных рядах (изменение волатильности, тренда или режима торгов). Для этого планируется использовать методы **обнаружения точек смены режима** (Change Point Detection).

Принцип работы [1]. Алгоритмы офлайн-обнаружения разбивают ряд на сегменты так, чтобы внутри каждого сегмента данные удовлетворяли выбранной модели (например, постоянное среднее или стационарное распределение), а в точках смены параметры модели менялись. Формально задаётся **функция потерь** (cost function) — сумма по сегментам «стоимости» подгонки модели на этом сегменте; **метод поиска** находит разбиение, минимизирующее суммарную стоимость при заданном числе точек смены (или с штрафом за их количество). Используются точные методы (динамическое программирование, перебор) или приближённые (жадные, бинарная сегментация). Рассматриваются как офлайн-методы (ретроспективный анализ всего ряда), так и онлайн-методы для потоковых данных.

Применение методов обнаружения точек смены режима в финансовых данных. В работе [2] проведена сравнительная оценка различных алгоритмов обнаружения точек смены режима применительно к финансовым временным рядам. Финансовые данные характеризуются наличием структурных изменений, вызванных экономическими шоками, изменениями в регуляторной политике, сменами рыночных режимов (от бычьего к медвежьему рынку и наоборот), а также изменениями в волатильности. Обнаружение таких точек смены режима критично для корректного моделирования и прогнозирования, так как применение единой модели ко всему временному ряду может привести к значительным ошибкам, если данные содержат несколько различных режимов.

Методы обнаружения точек смены режима в финансах применяются для сегментации исторических данных на периоды с различными статистическими свойствами (например, периоды высокой и низкой волатильности, трендовые и боковые движения), что позволяет адаптировать модели прогнозирования к текущему режиму рынка. В проекте предполагается применять офлайн-обнаружение для разметки режимов рынка и возможной сегментации данных при подготовке обучающих выборок, в том числе с использованием реализаций из библиотеки **ruptures**.

1.2 Рекуррентные нейронные сети (RNN, LSTM, GRU)

Рекуррентные сети предназначены для обработки последовательностей и учёта временной зависимости. Классическая архитектура RNN [3] на каждом шаге времени t обновляет скрытое состояние h_t на основе текущего входа x_t и предыдущего состояния h_{t-1} :

$$h_t = \varphi(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (1.1)$$

где φ — нелинейная функция активации (например, $\tanh$), W_{xh} , W_{hh} — матрицы весов, b_h — смещение. Выход на шаге t может вычисляться как $y_t = W_{hy}h_t + b_y$. Обучение выполняется методом обратного распространения во времени (BPTT). Недостаток простых RNN — затухание или взрыв градиента при длинных последовательностях, что затрудняет обучение долгосрочным зависимостям.

Долгая краткосрочная память (LSTM) [4] вводит ячейку памяти C_t и три воротых механизма (ворота забывания, входа и выхода), управляющих потоком информации. На каждом шаге t вычисляются: **ворота забывания** $f_t = \sigma(W_{xf}x_t + W_{hf}h_{t-1} + b_f)$ — доля информации из ячейки C_{t-1} , которую сохраняем; **ворота входа** $i_t = \sigma(W_{xi}x_t + W_{hi}h_{t-1} + b_i)$ и кандидат на обновление ячейки $\tilde{C}_t = \tanh(W_{xc}x_t + W_{hc}h_{t-1} + b_c)$; обновление ячейки $C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$ (звездочка — поэлементное умножение); **ворота выхода** $o_t = \sigma(W_{xo}x_t + W_{ho}h_{t-1} + b_o)$; скрытое состояние $h_t = o_t * \tanh(C_t)$. Функция σ — сигмоида, выходы ворот в $[0, 1]$, что позволяет стабильно передавать градиент через ячейку и смягчать проблему затухающего градиента.

Применение LSTM в финансовых временных рядах. Экспериментальные исследования показывают, что LSTM-модели превосходят традиционные методы временных рядов, такие как ARIMA, с существенным отрывом при прогнозировании финансовых данных [5]. В работе [5] проведено сравнительное исследование ARIMA, одностороннего LSTM и двунаправленного LSTM (BiLSTM) на данных акций IBM. Результаты демонстрируют, что глубокие нейронные сети с памятью, такие как LSTM, превосходят регрессионные подходы благодаря способности моделировать нелинейные зависимости и запоминать длинные последовательности входных данных через механизм ворот.

В задачах прогнозирования цен акций LSTM-сети применяются для предсказания будущих значений на основе исторических данных. Модель обучается на последовательностях фиксированной длины, где каждое окно содержит признаки за определённый период, и предсказывает значение цены на следующий временной шаг. Эксперименты [5] показывают, что LSTM-модели обрабатывают данные в одном направлении (слева направо), деля их на батчи размером 41-42 элемента, и достигают равновесия быстрее, чем двунаправленные варианты.

Двунаправленный LSTM (BiLSTM) в финансах. Архитектура BiLSTM расширяет возможности обычного LSTM за счёт дополнительного обучения данных в обратном направлении (справа налево), что позволяет модели учитывать как прошлые, так и будущие контексты при обработке каждого элемента последовательности [5]. Экспериментальные результаты [5] демонстрируют, что BiLSTM превосходит односторонний LSTM на 37.78% по снижению ошибок прогнозирования при работе с финансовыми временными рядами. Это улучшение достигается за счёт того, что модель обучается на данных дважды: сначала в прямом направлении (от прошлого к будущему), затем в обратном (от будущего к прошлому), что позволяет извлекать дополнительные паттерны из данных.

Интересной особенностью BiLSTM, выявленной в [5], является то, что модель делит те же данные на большее количество батчей (71-75 против 41-42 у обычного LSTM), используя меньшие размеры батчей. Это связано с тем, что BiLSTM должна обрабатывать данные в обоих направлениях, и длина обучающих данных, обрабатываемых каждым батчем, составляет примерно половину от длины батча в обычном LSTM. Кроме того,

BiLSTM достигает равновесия медленнее, чем односторонний LSTM, что указывает на способность модели извлекать дополнительные признаки из данных, которые не могут быть обнаружены при одностороннем обучении.

Сеть с блоками сброса ворот (GRU) [6] упрощает LSTM, используя два воротых блока. **Ворота сброса** $r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r)$ определяют, какую часть прошлого состояния учитывать при формировании кандидата. **Ворота обновления** $z_t = \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z)$ задают баланс между старым состоянием и новым. Кандидат скрытого состояния: $\tilde{h}_t = \tanh(W_{xh}x_t + (r_t * h_{t-1})W_{hh} + b_h)$. Итоговое состояние: $h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$. Таким образом, GRU объединяет идеи ворот забывания и входа LSTM в один механизм обновления; при меньшем числе параметров качество часто сопоставимо с LSTM, обучение быстрее.

Применение GRU в прогнозировании финансовой волатильности. В работе [7] предложена гибридная модель, объединяющая GARCH (Generalized Autoregressive Conditional Heteroskedasticity) и GRU для прогнозирования волатильности финансовых временных рядов. GARCH-модели традиционно используются для моделирования условной гетероскедастичности (изменяющейся во времени волатильности) в финансовых данных, но имеют ограничения при работе с нелинейными зависимостями. Интеграция GRU с GARCH позволяет комбинировать преимущества статистических моделей волатильности с способностью глубоких нейронных сетей улавливать сложные нелинейные паттерны. В такой гибридной архитектуре GARCH-компонент обеспечивает структурированное моделирование условной дисперсии, а GRU-компонент извлекает дополнительные нелинейные зависимости из остатков и других признаков, что улучшает точность прогнозирования волатильности по сравнению с использованием только GARCH или только GRU.

Рекуррентные сети с механизмом внимания для долгосрочного прогнозирования. Традиционные RNN, LSTM и GRU эффективны для одношагового и краткосрочного прогнозирования, но сталкиваются с трудностями при долгосрочном прогнозировании многомерных финансовых временных рядов [8]. Ключевая проблема долгосрочного прогнозирования заключается в необходимости одновременного захвата трёх типов зависимостей: пространственных корреляций между различными атрибутами в один момент времени, пространственно-временных отношений между различными атрибутами в разные моменты времени, и временных зависимостей между различными рядами [8].

Механизм внимания в RNN позволяет модели динамически определять веса различных входных признаков и временных шагов, что улучшает способность модели извлекать релевантную информацию из многомерных временных рядов. В работе [8] предложена архитектура DSTP-RNN (Dual-Stage Two-Phase RNN), вдохновлённая моделью человеческого внимания DSTP (Dual-Stage Two-Phase). Модель использует двухэтапную структуру внимания: на первом этапе производится сильный, но децентрализованный отклик весов внимания, а на втором этапе формируется стационарный и сконцентрированный отклик, что позволяет более точно моделировать пространственные корреляции между экзогенными рядами и целевым рядом.

В финансовом контексте DSTP-RNN применяется для долгосрочного прогнозирования многомерных временных рядов, где необходимо учитывать взаимосвязи между различными финансовыми инструментами, индикаторами и макроэкономическими фак-

торами. Модель использует множественные механизмы внимания на целевом ряде для усиления долгосрочных зависимостей, а также глубокий механизм пространственного внимания для извлечения сложных пространственных паттернов. Экспериментальные результаты [8] показывают, что attention-based RNN модели превосходят базовые методы на наборах данных в области финансов, энергии, окружающей среды и медицины, демонстрируя эффективность механизма внимания для долгосрочного прогнозирования многомерных временных рядов.

В проекте RNN, LSTM и GRU планируется использовать как базовые и сравнительные модели прогнозирования цен акций на MOEX, а также рассматривать варианты с механизмом внимания для улучшения долгосрочного прогнозирования.

1.3 Свёрточные нейронные сети для временных рядов

Свёрточные сети для одномерных временных рядов применяют **одномерную свёртку**: по оси времени скользит ядро (фильтр) длины k , на каждом сдвиге вычисляется скалярное произведение между фрагментом ряда и весами ядра плюс смещение, после чего применяется нелинейность. Таким образом извлекаются локальные паттерны (краткосрочные зависимости) в окне длины k . Несколько фильтров дают карту признаков; за свёрточными слоями могут следовать пулинг (усреднение или максимум по подпоследовательностям) для уменьшения размерности и агрегирования. В отличие от RNN, свёртки не имеют явного состояния между шагами и обрабатывают фиксированное окно; зато обучение параллелизуется по времени. Для прогноза ряда свёрточные блоки часто комбинируют с рекуррентными слоями или с полносвязными выходными слоями.

Применение CNN в прогнозировании финансовых временных рядов. В работе [9] исследуется применение свёрточных нейронных сетей для прогнозирования финансовых временных рядов. CNN-модели эффективно извлекают локальные паттерны и краткосрочные зависимости из временных рядов цен акций, используя одномерные свёрточные фильтры, которые скользят по временной оси. Это позволяет модели распознавать повторяющиеся паттерны, такие как технические фигуры, краткосрочные тренды и локальные аномалии в данных. В финансовом контексте CNN могут обрабатывать многомерные входные данные, включающие различные признаки (цены открытия, закрытия, максимумы, минимумы, объёмы), применяя свёртки как по временной оси, так и по оси признаков, что позволяет моделировать взаимосвязи между различными финансовыми показателями.

В работе [10] предложена архитектура **LSTNet** (Long- and Short-Term Time-series Network): сначала **свёрточный модуль** с одномерными фильтрами по времени извлекает краткосрочные локальные зависимости между переменными (при многомерном ряде — по всем каналам); затем выход свёрток подаётся в **рекуррентный модуль** (LSTM или GRU) для моделирования долгосрочных трендов; дополнительно может использоваться **авторегрессионный** линейный компонент для учёта масштаба ряда и устойчивости к нестационарности. В проекте планируется рассматривать как чисто свёрточные модели, так и гибриды CNN+RNN (в духе LSTNet) для прогноза цен акций.

1.4 Модели на основе механизма внимания (Transformer)

Архитектура **Transformer** [11] полностью основана на механизме **самовнимания** (self-attention) и не использует рекуррентные или свёрточные слои для кодирования порядка.

Это даёт параллелизацию по длине последовательности и явное моделирование связей между любыми двумя позициями.

Принцип работы. Для последовательности входных векторов формируются три набора представлений: **Query** (запрос) Q , **Key** (ключ) K , **Value** (значение) V — как линейные проекции входа: $Q = XW^Q$, $K = XW^K$, $V = XW^V$. Масштабированное скалярное внимание (scaled dot-product attention) вычисляется следующим образом: сначала считается произведение QK^T , делится на корень из размерности ключей, применяется softmax для получения весов внимания, которые умножаются на V . Матрица QK^T даёт оценки близости каждой позиции ко всем остальным; после softmax получаются веса внимания, которыми взвешиваются векторы V . Таким образом, каждая позиция обновляет своё представление как взвешенную сумму значений по всей последовательности. **Многоголовое внимание** (multi-head): несколько таких блоков с разными проекциями W^Q , W^K , W^V работают параллельно, их выходы конкатенируются и проецируются в итоговое представление. В полной архитектуре используются кодировщик и декодировщик из нескольких таких слоёв с остаточными связями и слоями нормализации; позиции элементов кодируются отдельно (позиционное кодирование), так как сам по себе механизм внимания не различает порядок.

Применение Transformer в прогнозировании финансовых временных рядов. В работе [9] исследуется применение архитектур Transformer для прогнозирования финансовых временных рядов. Механизм самовнимания в Transformer позволяет модели явно определять, какие исторические моменты времени наиболее релевантны для прогнозирования текущего значения, что особенно важно в финансовых данных, где различные временные периоды могут иметь различную значимость в зависимости от рыночных условий. Transformer-модели способны параллельно обрабатывать всю последовательность, что ускоряет обучение по сравнению с рекуррентными сетями, и могут моделировать долгосрочные зависимости без ограничений, связанных с затуханием градиента.

В финансовом контексте механизм внимания позволяет модели динамически фокусироваться на различных временных масштабах: краткосрочных флуктуациях, среднесрочных трендах и долгосрочных паттернах. Это особенно полезно для прогнозирования цен акций, где важно учитывать как недавние изменения цен, так и исторические паттерны, которые могут повторяться. Для адаптации к временным рядам и финансам применяют варианты с ограничением внимания по окну или специализированным позиционным кодированием, учитывающим временную структуру финансовых данных.

В проекте планируется использовать модели на основе Transformer (или их модификации для одномерных рядов) как один из классов кандидатов для прогнозирования цен акций на Московской бирже и сравнение с RNN/LSTM/GRU и CNN.

1.5 Формирование портфеля и оптимизация по риску

Для формирования инвестиционного портфеля на основе прогнозов модели планируется опираться на классическую постановку оптимизации по Марковицу [12]: минимизация дисперсии портфеля при заданной целевой доходности:

$$\text{минимум по } w \text{ величины } w^T \Sigma w \text{ при условиях } \mu^T w \geq r_{target} \text{ и } \sum_{i=1}^n w_i = 1,$$

где w — вектор весов активов, Σ — ковариационная матрица доходностей, μ — вектор ожидаемых доходностей. Ожидаемые доходности и ковариации могут оцениваться по историческим данным и/или с учётом прогнозов обученных моделей.

Известные ограничения модели Марковица — чувствительность к неточной оценке параметров и нестабильность весов. Для устойчивой оценки ковариационной матрицы применимы методы регуляризации, например, хорошо обусловленные оценки большой размерности [13]. В проекте планируется использовать подобные подходы при расчёте весов портфеля и ограничениях на риск и диверсификацию.

1.6 Метрики качества прогноза и эффективности стратегии

Качество моделей прогнозирования планируется оценивать метриками ошибки восстановления ряда: **MAE** (средняя абсолютная ошибка) и **RMSE** (среднеквадратичная ошибка). Для оценки эффективности торгового агента и сформированного портфеля используются финансовые метрики.

Коэффициент Шарпа [14]: $SR = \frac{R - r_f}{\sigma}$ — отношение избыточной доходности к стандартному отклонению (доходность на единицу риска).

Коэффициент Калмара [15]: $CR = \frac{R}{MDD}$ — отношение доходности к максимальной просадке (MDD).

При множественной проверке стратегий для корректировки за выборку применяется дефлированный коэффициент Шарпа [16]. Валидация и сравнение стратегий выполняются методом бэктестирования на исторических данных [17].

Применение методов машинного обучения к прогнозированию на российском фондовом рынке рассматривается, в частности, в [18]; в данной работе фокус делается на сочетании прогноза цен на МОЕХ, формирования портфеля и автоматизированного торгового агента с последующей оценкой по бэктесту.

1.7 Существующие модели предсказания прменных рядов, основанные на многоагентной симмуляции

В статье [19] изучается эффективность моделирования рынка через набор взаимодействующих между собой агентов (LLM). Статья особо интересна подходом к размышлению агента, который был выбран: Belief-Desire-Intention. Как следует из названия каждый агент руководствовался убеждениями, желаниями и немерениями при принятии решений. В совокупности с динамической системой обмена информацией между агентами, как было показано в статье, это дало возможность достоверно моделировать когнитивные искажения, социальные процессы. Например: самосбывающиеся пропущества, ммолелирует появление «пузырей», распространение паники. При тестировании на реальных новостях, графиках, система TwinMarket показала хорошие качественные прогнозы поведения рынка после реакции на новости, но В краткосрочной перспективе значительного успеха не было. Авторы даже не сравнивали свой алгоритм с стандартами вроде LSTM. Поэтому работа хотя и использует идею, похожую на нашу, у нее совершенно другое направление - она направлена не на предсказание точных цен для высокочастотной торговли, а на создание нового способа моделировать социологические эффекты рынка.

[20] Авторы рассматривают более общую задачу прогнозирования произвольных временных рядов. Можно считать, что предложенная архитектура MAA-TSF представляет собой ансамблирование над GAN в применении к прогнозированию временных рядов. Имеет место усложненная система обучения, характеризуемая объединением генераторов и дискриминаторов в динамическую систему, где агенты одновременно конкурируют через adversarial learning и кооперируются посредством механизмов дистилляции знаний и выравнивания распределений. Это позволяет не только смягчить типичные для GAN проблемы (схлопывание мод, нестабильность обучения), но и эффективно улавливать сложные, нестационарные паттерны в данных, что особенно ценно для волатильных рынков. Эксперименты демонстрируют превосходство MAA-TSF над базовыми GAN и ERM-моделями на множестве финансовых активов, хотя на устойчивых рядах выигрыш оказывается менее выраженным. Конкретно: Точность повышается с 55% для стандартных подходов (LSTM, LSTM + GAN) до 72%. Ключевое отличие от нашей работы - тот факт, что здесь многоагентность приеняется, как техника ансамблирования, влияющая на обучение, а в нашем случае многоагентность будет моделировать природу и устройство рынка. В частности, наша работа будет рассматривать частный случай временных рядов - образованные финансовыми инструментами

1.8 Инструменты анализа структуры временных рядов

В статье [21] авторы предлагают подход к анализу произвольных временных рядов через многоагентную систему, организованную по принципу «Анализатор–Мыслитель–Исполнитель». Главная инновация — использование визуально-языковой модели (VLM) для прямого «рассуждения» о графиках временных рядов. Критично, что ряд воспринимается не как последовательность данных, а как изображение, что приближает, как люди смотрят на данные.

Мыслитель (Reasoner) извлекает из изображения графика структурные «якоря» - ключевые точки, такие как пики, впадины, точки перегиба, - которые затем используются как семантические ограничения для реконструкции непрерывной траектории в латентном пространстве с помощью Neural ODE. Это позволяет модели оперировать не только числовыми паттернами, но и интуитивно понятной морфологией, которая явно представлена на визуализации.

Далее, в отличие от жёстких монолитных архитектур, система адаптивно составляет цепочки инструментов из предопределённой библиотеки (прогнозирование, декомпозиция, классификация и др.), выбирая оптимальную последовательность операций под конкретную задачу и характеристики данных. Это обеспечивает гибкость, аналогичную работе эксперта, который подбирает метод под ситуацию. Эксперименты на 21 наборе данных по задачам прогнозирования, классификации, импутации и обнаружения аномалий показали конкурентоспособность или превосходство MAS4TS над специализированными моделями и недавними подходами на основе больших языковых моделей. Например, в долгосрочном прогнозе на погодных данных система улучшила MSE на 7.9 – 11.5%, а в обнаружении аномалий достигла наивысшего среднего F_1 -score.

2 Модели прогнозирования доходности

В каждом из экспериментов: с разными видами кластеризации и без будет использоваться набор моделей, описанный ниже.

Каждая модель решает задачу регрессии: по скользящему окну длины T предсказать скалярную целевую переменную — **логарифмическую доходность** следующего шага:

$$r_{t+1} = \ln \frac{p_{t+1}}{p_t} \quad (2.2)$$

, за исключением ARIMA, который предсказывает логарифмическую доходность следующего шага: $\ln(p_{t+1})$, где p_t — цена закрытия последнего дня окна.

2.1 Подбор гиперпараметров

Для каждой модели реализован автоматический поиск гиперпараметров с использованием фреймворка Optuna, которая оптимизирует IC – Information Coefficient:

$$IC = \frac{\sum_i (y_i^{\text{true}} - \bar{y}^{\text{true}})(y_i^{\text{pred}} - \bar{y}^{\text{pred}})}{\sqrt{\sum_i (y_i^{\text{true}} - \bar{y}^{\text{true}})^2 \cdot \sum_i (y_i^{\text{pred}} - \bar{y}^{\text{pred}})^2}}. \quad (2.3)$$

В качестве метрики оптимизации IC был выбран по двум причинам:

1. Модели оптимизированные по IC обычно превосходят модели оптимизированные по MSE на задачах прогнозирования финансовых рядов
2. При низком IC классический Марковиц дает огромные ошибки в весах [22]

2.2 ARIMA

2.2.1 Архитектура

Модель ARIMA(p, d, q) [23] описывает временной ряд как стационарный процесс после d -кратного дифференцирования. AR-компонент моделирует зависимость от p лагов ряда, MA-компонент — от q лагов ошибки предсказания:

$$\varphi(B)(1 - B)^d y_t = \theta(B)\varepsilon_t, \quad (2.4)$$

где B — оператор сдвига, $\varphi(B) = 1 - \varphi_1 B - \dots - \varphi_p B^p$, $\theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$, $\varepsilon_t \sim \text{WN}(0, \sigma^2)$. Параметры оцениваются методом максимального правдоподобия. ARIMA принимает на вход одномерный лог-ценовой ряд $\ln(p_t)$ для достижения стационарности и предсказывает логарифмическую следующую член ряда $\ln(p_{t+1})$.

2.3 Реализация

Поскольку модель применяется лишь для прогона на всех акциях (без кластеризации), то для каждого тикера строится и оптимизируется своя модель, а затем каждая метрика усредняется по всем тикерам.

Для автоматического b и чесного выбора тройки (p, d, q) используется алгоритм **Auto ARIMA** [24], который итеративно перебирает комбинации параметров в заданных диапазонах, отбирая модель с минимальным значением информационного критерия AIC:

$$AIC = -2 \ln \hat{L} + 2k, \quad (2.5)$$

где $\hat{L}$ — максимум функции правдоподобия модели, k — число оцениваемых параметров.

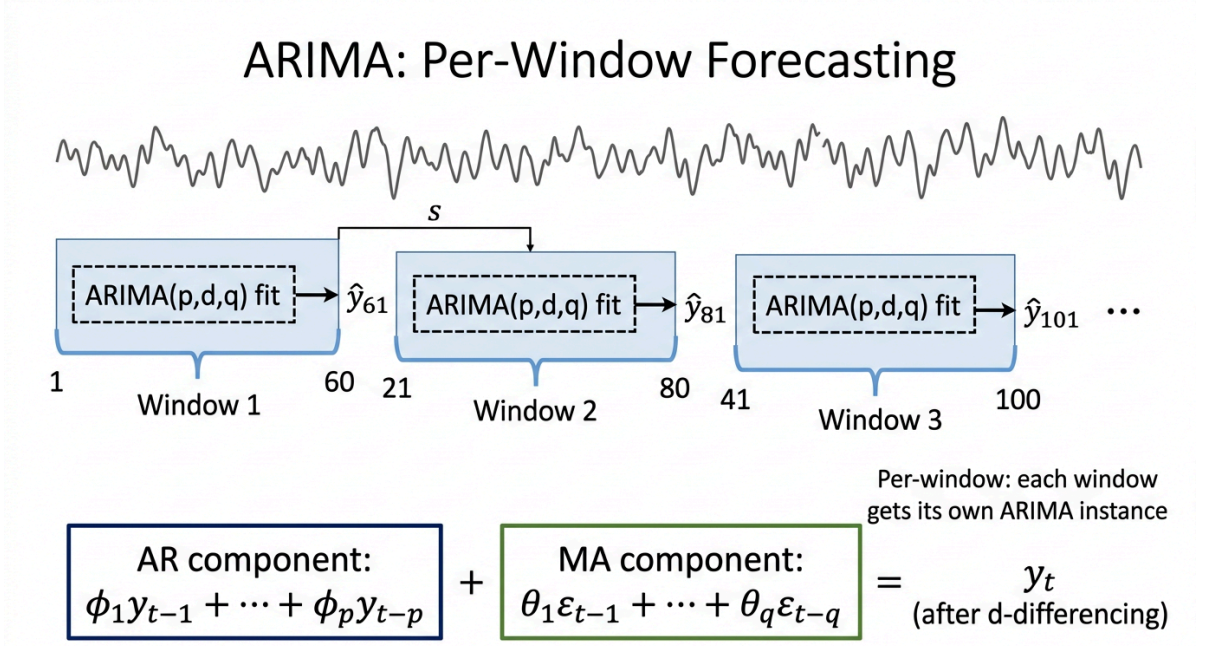


Рис. 2.1. Схема ARIMA в режиме per-window: каждое скользящее окно даёт независимый одношаговый прогноз.

Мы выбрали ARIMA в качестве бейзлайна, потому что ARIMA совершенно не требует GPU, что облегчает её тестирование в локальной среде, а также хорошо улавливает короткосрочные доходности. Однако Модель линейна — не улавливает нелинейные зависимости и кросс-признаковые взаимодействия. интерпретируема, а также не требует GPU и корректно улавливает краткосрочную автокорреляцию доходностей. Однако модель линейна, а значит не улавливает нелинейные зависимости, а финансовые ряды обычно обладают нелинейными свойствами, что деалет ARIMA менее конкурентоспособным.

2.4 LightGBM

2.4.1 Архитектура

LightGBM [25] — алгоритм градиентного бустинга на решающих деревьях или же GBDT с двумя ключевыми оптимизациями:

- leaf-wise рост дерева: вместо стандартного level-wise (разбиение всех листьев на одной глубине) выбирается и разбивается лист с наибольшим уменьшением функции потерь. Это позволяет строить более глубокие и точные деревья при том же числе листьев.
- Gradient-based One-Side Sampling: из обучающей выборки оставляются все наблюдения с крупными градиентами и случайная подвыборка с малыми, что ускоряет обучение без потери качества.

Ансамбль из M деревьев строится последовательно: дерево m аппроксимирует псевдо-остатки ансамбля из $m - 1$ деревьев:

$$\hat{y}^{\{(m)\}} = \hat{y}^{\{(m-1)\}} + \eta \cdot f_m(x), \quad (2.6)$$

где η — скорость обучения, f_m — m -е дерево. Финальный прогноз определяется как $\hat{y} = \sum_{m=1}^M \eta \cdot f_m(x)$.

2.4.2 Реализация

LightGBM с ходу не умеет работать с последовательностями, поэтому мы разворачиваем тензор (N, T, F) в матрицу $(N, T \cdot F)$, т.е. каждый временной шаг и каждый признак становятся отдельным столбцом — модель видит их как независимые числовые фичи без какого-либо порядка или памяти о последовательности.

Ранняя остановка осуществляется по валидационному L2.

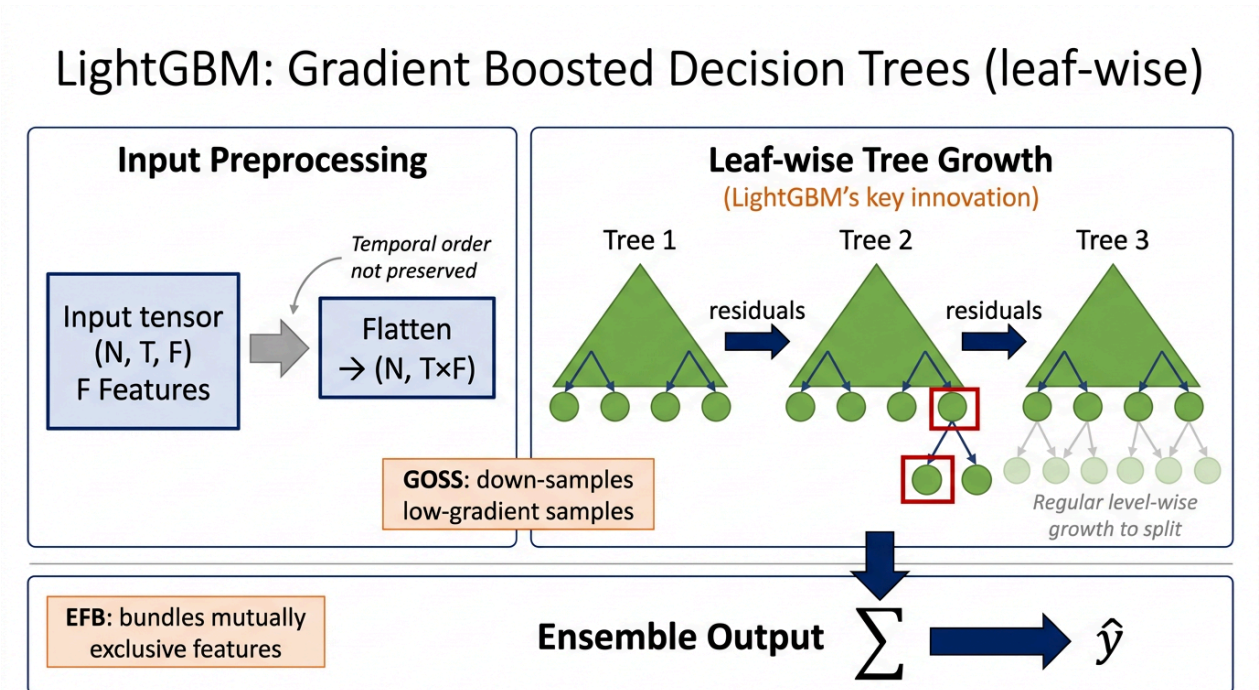


Рис. 2.2. Архитектура LightGBM: flatten временного ряда, leaf-wise рост деревьев, суммирование ансамбля.

Мы выбрали этот метод, потому что он устойчив к выбросам, хорошо работает на нелинейных взаимодействиях признаков, а также часто обходит нейросетевые модели. Ещё одним плюсом LightGBM является то, что он работает на CPU и достаточно быстро обучается даже на больших выборках.

Не смотря на все положительные свойства LightGBM, у него есть один значительный недостаток: ось времени разворачивается в плоский вектор — последовательности $x_1, \dots, x_T$ и $x_T, \dots, x_1$ для модели неразличимы без явных признаков позиции. Это делает LightGBM менее конкурентоспособным при сильных последовательных зависимостях [26].

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
n_estimators	Число деревьев в ансамбле	200–1200, шаг 100	int
learning_rate	Шаг сжатия η	0.005–0.2 (log)	float
num_leaves	Макс. листьев на дерево (leaf-wise)	15–255, шаг 16	int
max_depth	Макс. глубина (-1 = без ограничения)	-1 –14	int
min_child_samples	Мин. образцов в листе	5–80, шаг 5	int
subsample	Доля строк при построении дерева	0.5–1.0	float
colsample_bytree	Доля признаков при построении дерева	0.5–1.0	float
reg_alpha	L_1 -регуляризация весов листьев	0.0–1.0	float
reg_lambda	L_2 -регуляризация весов листьев	0.0–2.0	float

Таблица 2.1. Пространство поиска LightGBM. num_leaves — главный регулятор ёмкости при leaf-wise росте.

2.5 LSTM

2.5.1 Архитектура

Долгая краткосрочная память (Long Short-Term Memory, LSTM) [4] — рекуррентная архитектура, преодолевающая проблему затухающего градиента классических RNN. Ключевое нововведение — ячейка памяти C_t , отдельная от скрытого состояния h_t и управляемая тремя воротами:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad (2.7)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad \tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C), \quad (2.8)$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad (2.9)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad h_t = o_t \odot \tanh(C_t), \quad (2.10)$$

где σ — сигмоида, $\odot$ — поэлементное произведение. Ворота забывания f_t решает, какую долю предыдущей ячейки памяти сохранить; ворота входа i_t контролирует запись нового кандидата $\tilde{C}_t$; ворота выхода o_t определяет, что передать в скрытое состояние.

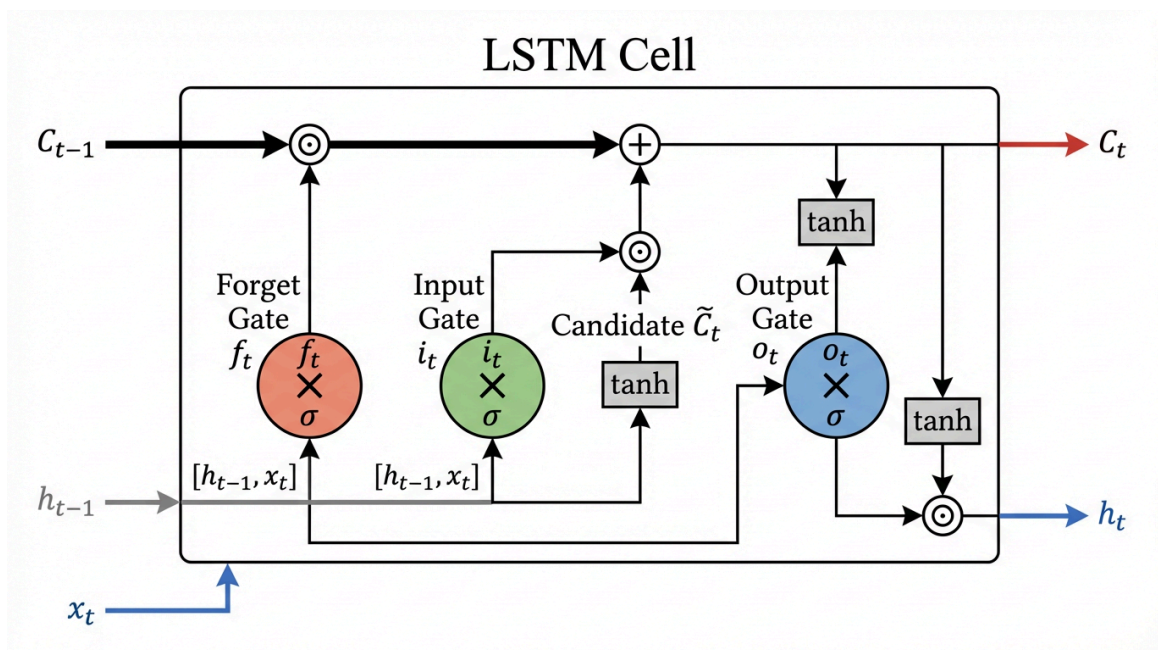


Рис. 2.3. Структура LSTM-ячейки: ячейка памяти C_t (горизонтальный поток) защищена тремя воротами f_t , i_t , o_t .

2.5.2 Реализация

Стек из `num_layers` LSTM-слоёв с размером скрытого состояния `hidden_size`; dropout применяется между слоями при `num_layers > 1`. Скрытое состояние последнего шага h_T подаётся в линейный слой $h_T \rightarrow \hat{y}$. Обучение — Adam, потеря — MSE, ранняя остановка по валидационной потере.

Мы выбрали LSTM, потому что гейтовый механизм позволяет выборочно «запоминать» медленно меняющиеся тренды и «забывать» устаревшую информацию — важное свойство для финансовых рядов со сменой режимов. Работа [27] показала, что LSTM на дневных данных S&P 500 статистически значимо превосходит случайный лес по Directional Accuracy, а в [5] LSTM устойчиво превосходит ARIMA по MAE и RMSE.

Несмотря на это, LSTM имеет существенные ограничения: последовательная обработка не параллелизуется, при длинных окнах информация начала может «вымываться» даже с гейтами, а высокое число параметров повышает риск переобучения.

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
<code>hidden_size</code>	Размер скрытого состояния h_t	32–256, шаг 32	int
<code>num_layers</code>	Число стековых LSTM-слоёв	1–3	int
<code>dropout</code>	Dropout между слоями	0.0–0.4	float
<code>lr</code>	Скорость обучения Adam	1e-4–5e-3 (log)	float
<code>batch_size</code>	Размер батча	{32, 64, 128}	cat
<code>epochs</code>	Максимум эпох	20–80, шаг 10	int
<code>patience</code>	Порог ранней остановки	5–15	int

Таблица 2.2. Пространство поиска LSTM (50 проб).

2.6 GRU

2.6.1 Архитектура

Управляемый рекуррентный блок (Gated Recurrent Unit, GRU) [6] упрощает LSTM, объединяя ворота забывания и входа в единое **ворота обновления** z_t и вводя **ворота сброса** r_t :

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r), \quad (2.11)$$

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z), \quad (2.12)$$

$$\tilde{h}_t = \tanh(W[h_t \odot h_{t-1}, x_t] + b), \quad (2.13)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad (2.14)$$

Ворота сброса r_t определяет, сколько предыдущего состояния учитывать при формировании кандидата; ворота обновления z_t задаёт баланс между старым и новым. Ячейка памяти C_t отсутствует — скрытое состояние h_t выполняет обе роли.

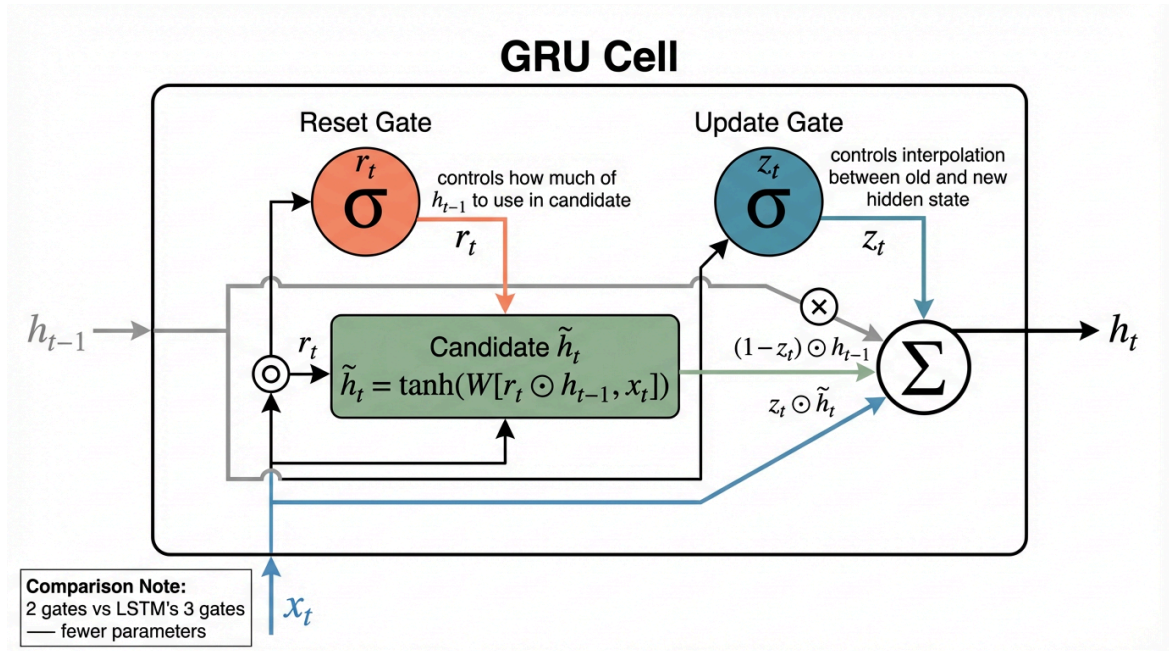


Рис. 2.4. Структура GRU-ячейки: два ворота вместо трёх, нет отдельной ячейки памяти — меньше параметров, чем у LSTM.

2.6.2 Реализация

Архитектура идентична LSTM: стек из `num_layers` GRU-слоёв, dropout между ними, скрытое состояние последнего шага h_T подаётся в линейный слой. Обучение — Adam, потеря — MSE, ранняя остановка по валидационной потере.

Мы выбрали GRU как более эффективную альтернативу LSTM: 25% меньше параметров при аналогичном качестве на коротких последовательностях ($T \leq 100$). Исследование [5] показало схожую точность GRU и LSTM на финансовых данных, однако GRU сходил быстрее. На датасетах среднего размера GRU нередко является более практичным выбором [26].

Те же структурные ограничения последовательного вычисления, что и у LSTM. При $T = 60$ отсутствие явной ячейки памяти не является критичным.

Подбор гиперпараметров (50 проб Optuna). Пространство поиска идентично LSTM — меньший объём модели при том же бюджете проб позволяет GRU быстрее находить оптимум.

Параметр	Описание	Диапазон	Тип
hidden_size	Размер скрытого состояния h_t	32–256, шаг 32	int
num_layers	Число стековых GRU-слоёв	1–3	int
dropout	Dropout между слоями	0.0–0.4	float
lr	Скорость обучения Adam	1e-4–5e-3 (log)	float
batch_size	Размер батча	{32, 64, 128}	cat
epochs	Максимум эпох	20–80, шаг 10	int
patience	Порог ранней остановки	5–15	int

Таблица 2.3. Пространство поиска GRU (50 проб).

2.7 CNN

2.7.1 Архитектура

Одномерная свёрточная нейронная сеть применяет ядро размера k вдоль временной оси, обнаруживая локальные паттерны независимо от их позиции в окне. Операция свёртки для i -й позиции и j -го фильтра:

$$(X * W)_{i,j} = \sum_{c=1}^F \sum_{\tau=0}^{k-1} X_{i+\tau,c} \odot W_{\tau,c,j} + b_j, \quad (2.15)$$

где $W \in \mathbb{R}^{k \times F \times C}$ — тензор весов, C — число фильтров, k — размер ядра. Дополнение `padding = k // 2` сохраняет длину временной оси.

2.7.2 Реализация

Тензор (batch, T, F) транспонируется в (batch, F, T) для PyTorch Conv1d. Проходит num_layers блоков:

$$\text{Block} : \text{Conv1d}(F \rightarrow C, k) \rightarrow \text{BatchNorm1d}(C) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(p). \quad (2.16)$$

AdaptiveAvgPool1d(1) агрегирует временную ось и линейный слой формирует прогноз.

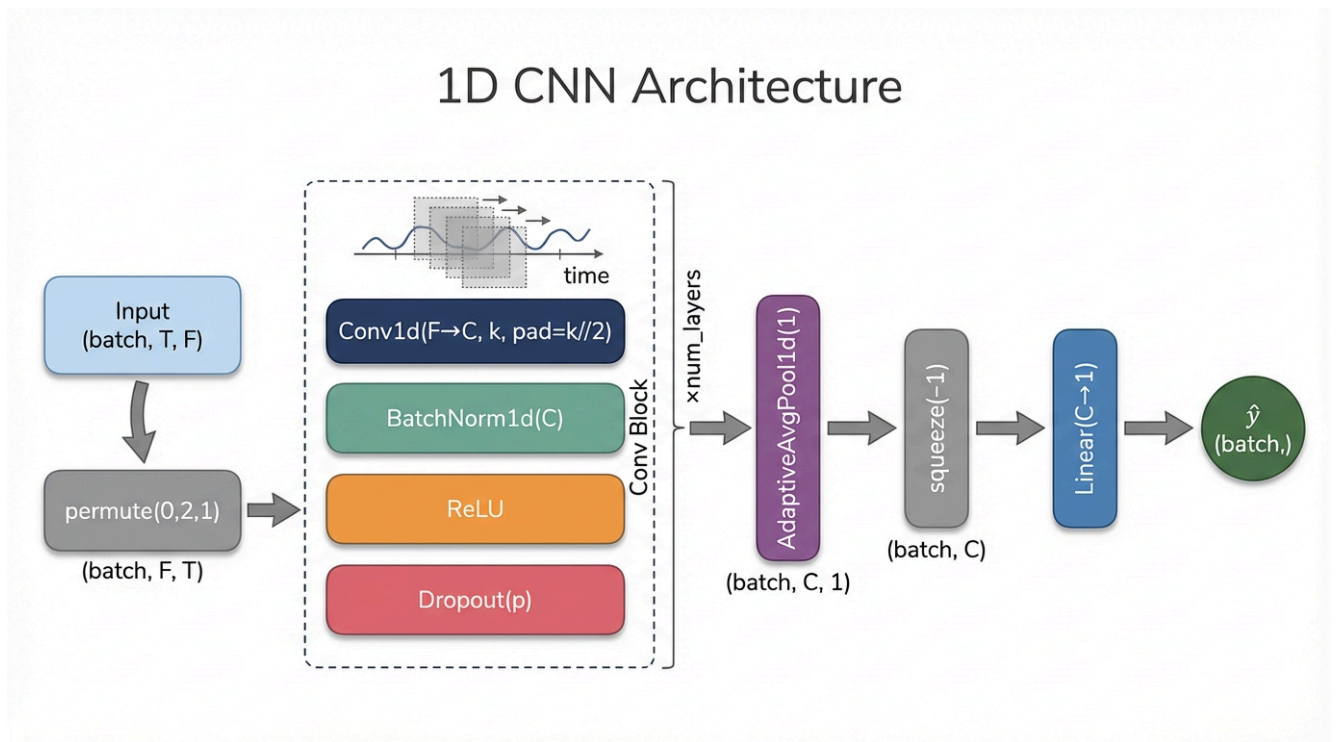


Рис. 2.5. Архитектура 1D CNN: транспонирование, блоки Conv1d→BN→ReLU→Dropout, глобальный Average Pooling, линейный выход.

Мы выбрали CNN, потому что свёртки с различными `kernel_size` улавливают локальные паттерны разных временных масштабов ($k = 3 \approx$ неделя, $k = 9 \approx$ полмесяца), а вычисления полностью параллелизованы — CNN обучается значительно быстрее LSTM/GRU. `BatchNorm` стабилизирует обучение на нестационарных данных [26].

Рецептивное поле ограничено $L = \text{num_layers} \times k$, поэтому длинные зависимости CNN улавливает хуже рекуррентных моделей. `AdaptiveAvgPool1d` усредняет все позиции — стирается информация о том, **когда** паттерн произошёл.

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
<code>num_filters</code>	Число свёрточных фильтров C	32–256, шаг 32	int
<code>num_layers</code>	Число свёрточных блоков	1–3	int
<code>kernel_size</code>	Размер ядра k (временной масштаб паттерна)	{3, 5, 7, 9}	int
<code>dropout</code>	Dropout в каждом блоке	0.0–0.4	float
<code>lr</code>	Скорость обучения Adam	1e-4–5e-3 (log)	float
<code>batch_size</code>	Размер батча	{32, 64, 128}	cat
<code>epochs</code>	Максимум эпох	20–80, шаг 10	int
<code>patience</code>	Порог ранней остановки	5–15	int

Таблица 2.4. Пространство поиска CNN. `kernel_size` задаёт временной масштаб: $k = 3 \approx$ неделя, $k = 9 \approx$ полмесяца.

2.8 Transformer

2.8.1 Архитектура

Transformer [11] отказывается от рекуррентности, заменяя её механизмом **самовнимания** (self-attention), который явно моделирует связи между любыми двумя позициями последовательности.

Входные признаки проецируются в пространство d_{model} , затем к каждой позиции прибавляется синусоидальное позиционное кодирование:

$$\text{PE}_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{2\frac{i}{d_{\text{model}}}}}\right), \quad \text{PE}_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10000^{2\frac{i}{d_{\text{model}}}}}\right). \quad (2.17)$$

Ядро каждого энкодер-слоя — **масштабированное dot-product внимание**:

$$\text{Attention}(Q, K, V) = \text{softmax}\frac{QK^T}{\sqrt{d_k}}V, \quad (2.18)$$

где $Q = XW^Q$, $K = XW^K$, $V = XW^V$. **Многоголовое внимание** (МНА) с n_{head} головами конкатенирует результаты параллельных голов:

$$\text{МНА}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_{n_{\text{head}}})W^O, \quad \text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V) \quad (2.19)$$

За МНА следует FFN: $\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$. Оба блока охвачены остаточными связями и нормализацией (Add & Norm).

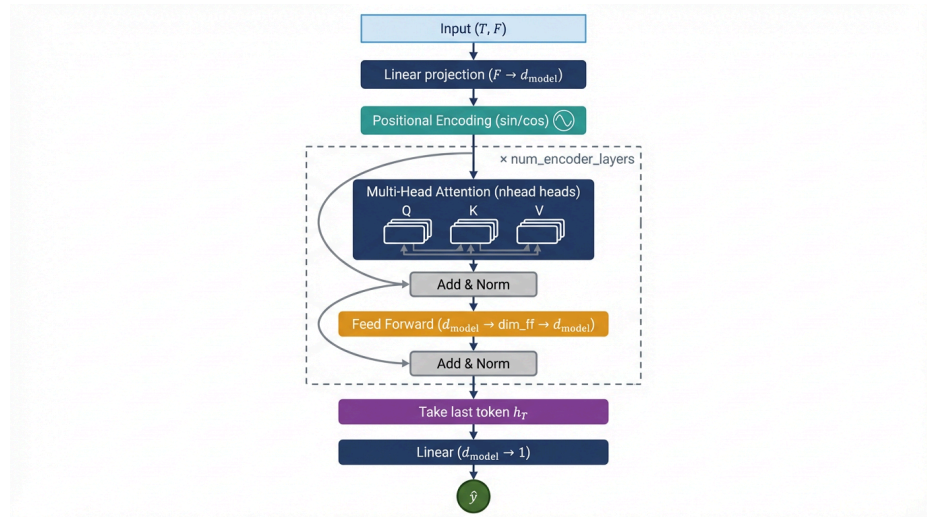


Рис. 2.6. Архитектура Transformer-кодировщика: проекция, позиционное кодирование, стек МНА+FFN с остаточными связями.

2.8.2 Реализация

Используется энкодер-only Transformer. Прогноз формируется по выходу последнего токена h_T , который подаётся в линейный слой. Обучение — Adam, потеря — MSE, ранняя остановка по валидационной потере.

Мы включили Transformer, потому что МНА позволяет модели явно связать любые два момента в окне без «вымывания» информации, характерного для RNN — потенциа-

но ценное свойство для событийных реакций (дивидендные даты, earnings). Вычисления полностью параллелизованы [28].

Однако работа [29] показывает, что на большинстве задач прогнозирования временных рядов простая линейная модель превосходит Transformer-архитектуры. На коротких финансовых окнах ($T = 60$) модель склонна к переобучению из-за квадратичного роста сложности $O(T^2)$ и большого числа параметров [26].

Подбор гиперпараметров (50 проб Optuna).

Параметр	Описание	Диапазон	Тип
d_model	Размерность эмбединга токена	32–256, шаг 32	int
nhead	Число голов внимания (делитель d_model)	{2, 4, 8}	cat
num_encoder_layers	Число слоёв кодировщика	1–4	int
dim_feedforward	Размерность скрытого слоя FFN	64–512, шаг 64	int
dropout	Dropout в МНА и FFN	0.0–0.4	float
lr	Скорость обучения Adam	1e-4–5e-3 (log)	float
batch_size	Размер батча	{32, 64, 128}	cat
epochs	Максимум эпох	20–80, шаг 10	int
patience	Порог ранней остановки	5–15	int

Таблица 2.5. Пространство поиска Transformer. nhead ограничен делимостью d_model.

2.9 Метрики оценки

Качество прогноза оценивается набором метрик на тестовой выборке. Здесь y_i — истинная логарифмическая доходность, $\hat{y}_i$ — предсказанная, n — размер выборки.

MAE (Mean Absolute Error) — средняя абсолютная ошибка, интерпретируемая в единицах доходности:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (2.20)$$

RMSE (Root Mean Squared Error) — среднеквадратичная ошибка, штрафующая крупные отклонения сильнее, чем MAE:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (2.21)$$

MAPE (Mean Absolute Percentage Error) — средняя абсолютная процентная ошибка; нестабильна при $y_i \approx 0$, поэтому в знаменатель добавляется $\varepsilon = 10^{-8}$:

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i| + \varepsilon}. \quad (2.22)$$

R^2 (коэффициент детерминации) — доля дисперсии целевой переменной, объяснённая моделью; $R^2 = 1$ соответствует идеальному прогнозу, $R^2 = 0$ — уровню наивного прогноза средним:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}. \quad (2.23)$$

Directional Accuracy (DA) — доля шагов, на которых знак прогнозируемой доходности совпадает с реальным. Случайное угадывание соответствует $DA = 0.5$; значение выше 0.5 означает, что модель правильно предсказывает направление движения цены:

$$DA = \frac{1}{n} \sum_{i=1}^n \mathbb{1}[\text{sign}(y_i) = \text{sign}(\hat{y}_i)]. \quad (2.24)$$

IC (Information Coefficient) — коэффициент корреляции Пирсона между прогнозами и реальными доходностями. Это основная метрика выбора модели, поскольку для формирования портфеля критична ранговая согласованность прогнозов, а не абсолютная ошибка: модель с высоким RMSE, но ненулевым IC, всё равно несёт ценный торговый сигнал. В количественных финансах $IC > 0.05$ считается значимым результатом:

$$IC = \frac{\sum_i (y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_i (y_i - \bar{y})^2 \cdot \sum_i (\hat{y}_i - \bar{\hat{y}})^2}}. \quad (2.25)$$

IC предпочтителен перед RMSE для портфельных задач: оптимизация по Марковицу [\[12\]](#) опирается на ожидаемые доходности, и модель с ненулевым IC способна улучшить качество весов портфеля даже при значительной абсолютной ошибке прогноза.

3 Архитектура пайплайна эмбедингов и кластеризации

Предлагаемый в работе подход к анализу и последующему прогнозированию цен акций на Московской бирже базируется на идее перехода от непосредственной работы с сырыми временными рядами к работе с их латентными представлениями — эмбедингами. Основная гипотеза заключается в том, что поведение ценового графика на ограниченном временном горизонте (окне) может быть закодировано в некоторое векторное представление фиксированной размерности, причём близость таких представлений в латентном пространстве будет отражать схожесть динамики цены вне зависимости от абсолютных значений и конкретного инструмента. Гипотеза об эффективности конкретных инструментов свободится на практике к тому, что мы рассматриваем рынок - как структуру динамическую. Паттерны могут менять год от к году и те, что давали эффекты в прошлом могут быть убыточны сейчас. В то же время мы верим, что «Ничто не ново под луной». На практике это означает, что новые алгоритмы и способы рассмотрения рынка появляются редко. Поэтому то, что для конкретного временного ряда может являться новым паттерном - могло встречаться ранее в других временных рядах (например, в других акциях).

Применение эмбедингов позволяет, во-первых, абстрагироваться от масштаба цен отдельных акций, а во-вторых, выявлять повторяющиеся паттерны поведения, которые могут соответствовать определённым рыночным режимам или фазам движения.

Пайплайн обработки данных и построения эмбедингов состоит из нескольких последовательных этапов, каждый из которых вносит вклад в формирование итогового латентного пространства и последующей интерпретации результатов. На вход всей системы поступают сырые данные по акциям Московской биржи, включающие стандартные для анализа финансовых временных рядов параметры: цены открытия, закрытия, максимумы, минимумы и объёмы торгов. Важно отметить, что мы не отбираем только ликвидные акции - если бы мы так поступили, был бы велик шанс столкнуться с «look-ahead bias» - ошибкой, связанной с выбором акций, которые достоверно будут пользоваться популярностью и с большей вероятностью будут расти, что в свою очередь позволит быть прибыльными тем алгоритмам, которые на данных без этого смещения, эффективным не будут.

3.1 Выбор данных

Первым этапом является предварительная фильтрация и нормализация данных. Для каждого тикера выполняется очистка от пропусков и выбросов, после чего ценовые ряды приводятся к сопоставимому виду. В качестве основной техники нормализации используется приведение к логарифмической доходности либо масштабирование относительно локального окна, что позволяет нейтрализовать эффект различий в абсолютных ценах между разными акциями и сосредоточиться на форме движения, а не на его амплитуде. Такой подход особенно важен для последующей кластеризации, поскольку без нормализации бумаги с высокой ценой (например, акции Сбербанка) будут доминировать в пространстве признаков исключительно за счёт масштаба, а не за счёт характерной динамики.

Второй этап — формирование скользящих окон фиксированной длины. Каждое окно представляет собой непрерывную последовательность из (L) временных шагов (свечей), для которых известны все необходимые признаки. Выбор длины окна является

критическим гиперпараметром и в данной работе обосновывается как компромисс между двумя факторами. С одной стороны, окно должно быть достаточно длинным, чтобы захватывать значимые паттерны, такие как краткосрочные тренды, развороты, консолидации или пробои уровней. С другой стороны, слишком длинное окно приводит к размыванию локальной структуры и увеличению вычислительной сложности.

путем перебора гиперпараметров было выбрано значение $L = 60$, что соответствует примерно десяти торговым дням при часовом таймфрейме или двум месяцам при дневном. Это окно, с одной стороны, достаточно для проявления типичных технических паттернов, а с другой — не настолько велико, чтобы включать несколько разнородных рыночных режимов одновременно, что усложнило бы задачу эмбединга.

3.2 Обучение сети

Третий этап — кодирование каждого окна в эмбединг фиксированной размерности с помощью рекуррентной нейронной сети. В качестве архитектуры кодировщика выбрана сеть на основе LSTM, что обусловлено способностью данного класса моделей удерживать долгосрочные зависимости в последовательности и при этом избегать затухания градиента, характерного для простых рекуррентных сетей. Архитектура кодировщика устроена следующим образом: на вход подаётся последовательность признаков окна размерности $(L \times d)$, где (d) — число признаков на одну свечу (например, OHLCV даёт $(d=5)$). Далее последовательно проходимся один или несколько слоёв LSTM, причём на последнем шаге скрытое состояние либо объединяется с состоянием ячейки, либо используется выход последнего временного шага в качестве агрегированного представления. В результате для каждого окна получается вектор $(e \in \mathbb{R}^h)$, где (h) — размерность эмбединга, задаваемая как гиперпараметр (в эксперименте использовалось значение $(h=64)$).

Обучение кодировщика осуществляется в автоэнкодерной постановке или с использованием контрастивной цели, когда близкие по динамике окна должны получать близкие эмбединги, а далёкие — удалённые. Важно отметить, что кодировщик обучается на всей совокупности окон по всем выбранным акциям, что позволяет ему вырабатывать универсальные представления, инвариантные к конкретному инструменту. После завершения обучения кодировщик фиксируется, и все окна пропускаются через него для получения эмбедингов, которые затем становятся объектами кластеризации.

Четвёртый этап — кластеризация полученных эмбедингов в латентном пространстве. Цель кластеризации — выделить группы окон, демонстрирующих схожее ценовое поведение, независимо от того, какой акции они принадлежат и в какой временной период они наблюдались. В качестве алгоритма кластеризации в работе используется (k) -means с последующим подбором оптимального числа кластеров по силуэтному коэффициенту и индексу Дэвиса–Боулдина. Альтернативно рассматривается иерархическая кластеризация, позволяющая визуализировать структуру сходства между эмбедингами в виде дендрограммы. Результатом этапа является разметка каждого окна меткой кластера, которая интерпретируется как принадлежность к определённому типу ценовой динамики.

Завершающим этапом пайплайна является анализ выделенных кластеров и их привязка к содержательным рыночным режимам. Для каждого кластера вычисляются статистические характеристики входящих в него окон: средняя волатильность, характерный наклон тренда, типичная амплитуда движения, а также временн'а и инструментальная

структура. Это позволяет дать интерпретацию кластерам, например, выделить кластеры, соответствующие импульсному росту, боковой консолидации, высоковолатильным разнонаправленным движениям или плавному нисходящему тренду. Интерпретируемость кластеров критически важна для последующего этапа формирования портфеля и принятия торговых решений: если агент знает, что текущее окно относится к кластеру «высокая волатильность без тренда», он может снизить размер позиции или переключиться на менее рискованные активы.

Предложенный пайплайн позволяет перейти от хаотичного множества ценовых графиков к структурированному латентному пространству, в котором выделяются повторяющиеся поведенческие паттерны. Это создаёт основу для двух ключевых применений в рамках проекта. Во-первых, кластеризация эмбедингов может использоваться для адаптивного выбора прогнозной модели: в зависимости от того, к какому кластеру относится текущее окно, может применяться специализированная модель, обученная на окнах из того же кластера. Во-вторых, информация о кластерах может быть интегрирована в процесс формирования портфеля, например, путём ограничения или расширения набора рассматриваемых активов в зависимости от того, в каком режиме они находятся. Таким образом, пайплайн эмбедингов и кластеризации выступает не как самоцель, а как фундамент для построения более интеллектуальной и адаптивной торговой системы.

3.3 Работа с данными

3.4 Данные

В работе используются дневные данные о свечах всех валидных акций Московской биржи. Свечи базово включают 5 основных параметров помимо timestamp:

open	high	low	close	volume
------	------	-----	-------	--------

Где **open** - цена открытия свечи, **high** - максимальная цена свечи, **low** - минимальная цена свечи, **close** - цена закрытия свечи, **volume** - объём торгов.

Эти данные были собраны за период с начала существования акции на бирже вплоть до 2026 года.



Рис. 3.7. Дневные свечи акции SBER

3.5 Предобработка данных и создание признаков

Перед добавлением признаков в данные были удалены пропуски, а также исключены все акции, у которых было менее 252 торговых дней.

Далее для каждого временного ряда были добавлены следующие признаки:

sma5	sma20
ema12	ema26
close_sma20	macd
macd_signal	macd_hist
rsi14	bb_pct
bb_bw	atr14
obv	

Где признаки вычисляются по формулам:

- $sma5_t = \frac{1}{5} \sum_{i=0}^4 close_{t-i}$ - средняя цена за 5 дней
- $sma20_t = \frac{1}{20} \sum_{i=0}^{19} close_{t-i}$ - средняя цена за 20 дней
- $ema12_t = \alpha_{12} close_t + (1 - \alpha_{12})ema12_{t-1}$, $\alpha_{12} = \frac{2}{12+1}$ - экспоненциальная средняя цена за 12 дней
- $ema26_t = \alpha_{26} close_t + (1 - \alpha_{26})ema26_{t-1}$, $\alpha_{26} = \frac{2}{26+1}$ - экспоненциальная средняя цена за 26 дней
- $close_sma20_t = \frac{close_t}{sma20_t}$ - отношение цены к средней цене за 20 дней
- $macd_t = ema12_t - ema26_t$ - разница между экспоненциальными средними за 12 и 26 дней
- $macd_signal_t = ema9(macd_t)$ - экспоненциальная скользящая средняя за 9 дней от $macd_t$
- $macd_hist_t = macd_t - macd_signal_t$ - разница между $macd_t$ и $macd_signal_t$

- $rsi14_t = 100 - \frac{100}{1+RS_t}$, $RS_t = \frac{avgGain14_t}{avgLoss14_t}$ - индекс относительной силы за 14 дней
- $sigma20_t = std(close_{t-19}, \dots, close_t)$ - стандартное отклонение цены за 20 дней
- $bb_upper_t = sma20_t + 2 \times sigma20_t$, $bb_lower_t = sma20_t - 2 \times sigma20_t$ - верхняя и нижняя границы полос Боллинджера
- $bb_pct_t = \frac{close_t - bb_lower_t}{bb_upper_t - bb_lower_t}$ - относительное положение цены внутри полос Боллинджера
- $bb_bw_t = \frac{bb_upper_t - bb_lower_t}{sma20_t}$ - ширина полос Боллинджера
- $TR_t = \max(high_t - low_t, abs(high_t - close_{t-1}), abs(low_t - close_{t-1}))$ - истинный диапазон
- $atr14_t = ema14(TR_t)$ - экспоненциальная средняя истинного диапазона за 14 дней
- $obv_t = obv_{t-1} + sign(close_t - close_{t-1}) \times volume_t$ - накопленный баланс объема торгов

Где $close_t$, $high_t$, low_t , $volume_t$ — значения цены закрытия, максимума, минимума и объема в день t соответственно.

Таким образом, всего получается 18 признаков, на которых обучаются модели.

open	high
low	close
volume	sma5
sma20	ema12
ema26	close_sma20
macd	macd_signal
macd_hist	rsi14
bb_pct	bb_bw
atr14	obv

Все данные были разделены на три непересекающихся временных выборки: тренировочную, валидационную и тестовую. Разбиение выполнено по дате, а не случайно, чтобы исключить утечку данных из будущего в прошлое.

Поскольку часть акций появилась на MOEX лишь в 2014–2015 годах, тренировочная выборка охватывает наибольший временной промежуток.

Выборка	Период	Доля
Train	2000 — 2022	5/7
Validation	2023 — 2024	1/7
Test	2025 — март 2026	1/7

.

3.6 Методы построения эмбендингов

Так как кластеризация временных окон становится доступной только после приведения их к единому векторному формату. Поэтому на оконных данных обучаются эмбендинги - числовые представления оконных данных, сохраняющие информацию о структуре и динамике временного ряда.

Для построения эмбендингов были использованы следующие методы:

- Handmade Embedder - метод построения эмбендингов на основе ручных признаков
- TS2Vec Embedder - метод построения эмбендингов на основе TS2Vec модели

3.7 Handmade Embedder

Handmade Embedder - метод построения эмбендингов на основе ручных признаков.

Handmade Embedder использует следующие признаки:

- Close Price
- High Price
- Low Price
- Volume
- Open Price

Список литературы

- [1] C. Truong, L. Oudre, и N. Vayatis, «Selective review of offline change point detection methods», *arXiv preprint*, 2018.
- [2] «An Evaluation of Change Point Detection Algorithms», 2024.
- [3] J. L. Elman, «Finding Structure in Time», *Cognitive Science*, т. 14, вып. 2, сс. 179–211, 1990, doi: [10.1207/s15516709cog1402_1](https://doi.org/10.1207/s15516709cog1402_1).
- [4] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, и J. Schmidhuber, «LSTM: A Search Space Odyssey», *IEEE Transactions on Neural Networks and Learning Systems*, т. 28, вып. 10, сс. 2222–2232, 2017.
- [5] S. Siami-Namini, N. Tavakoli, и A. Siami Namin, «A Comparative Analysis of Forecasting Financial Time Series Using ARIMA, LSTM, and BiLSTM», 2019.
- [6] K. Cho, B. van Merriënboer, Ç. Gülçehre, F. Bougares, H. Schwenk, и Y. Bengio, «Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation», *arXiv preprint*, 2014.
- [7] «Integrated GARCH-GRU in Financial Volatility Forecasting», 2024.
- [8] Y. Liu, C. Gong, L. Yang, и Y. Chen, «DSTP-RNN: a dual-stage two-phase attention-based recurrent neural networks for long-term and multivariate time series prediction», 2019.
- [9] «Financial Time Series Forecasting using CNN and Transformer», 2024.
- [10] G. Lai, W.-C. Chang, Y. Yang, и H. Liu, «Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks», *arXiv preprint*, 2017.
- [11] A. Vaswani и др., «Attention Is All You Need», *arXiv preprint*, 2017.
- [12] H. Markowitz, «Portfolio Selection», *The Journal of Finance*, т. 7, вып. 1, сс. 77–91, 1952.
- [13] O. Ledoit и M. Wolf, «A well-conditioned estimator for large-dimensional covariance matrices», *Journal of Multivariate Analysis*, т. 88, вып. 2, сс. 365–411, 2004.
- [14] W. F. Sharpe, «The Sharpe Ratio», *The Journal of Portfolio Management*, т. 21, вып. 1, сс. 49–58, 1994.

- [15] T. W. Young, «Calmar Ratio: A Smoother Tool», *Futures Magazine*, т. 20, вып. 1, 1991.
- [16] D. H. Bailey и M. López de Prado, «The Deflated Sharpe Ratio: Correcting for Selection Bias», *The Journal of Portfolio Management*, т. 40, вып. 5, сс. 94–107, 2014.
- [17] R. Pardo, *The Evaluation and Optimization of Trading Strategies*, 2nd изд. Wiley, 2008.
- [18] Е. А. Федорова и В. А. Черепенко, «Применение методов машинного обучения для прогнозирования доходности акций на российском фондовом рынке», *Экономический журнал ВШЭ*, т. 24, вып. 4, сс. 560–590, 2020.
- [19] Y. Yang и др., «TwinMarket: A Scalable Behavioral and Social Simulation for Financial Markets». [Онлайн]. Доступно на: <https://arxiv.org/abs/2502.01506>
- [20] Y. Qiao и др., «Multi-Agent Adversarial Time Series Forecasting». [Онлайн]. Доступно на: <https://openreview.net/forum?id=lj1qwU6Gxs>
- [21] W. Ruan и Y. Liang, «Visual Reasoning over Time Series via Multi-Agent System». [Онлайн]. Доступно на: <https://arxiv.org/abs/2602.03026>
- [22] R. Kan и G. Zhou, «Optimal Portfolio Choice with Parameter Uncertainty», *Journal of Financial and Quantitative Analysis*, т. 42, вып. 3, сс. 621–656, 2007, doi: [10.1017/S0022109000004129](https://doi.org/10.1017/S0022109000004129).
- [23] G. E. P. Box и G. M. Jenkins, «Time Series Analysis: Forecasting and Control», *Holden-Day*, 1970.
- [24] R. J. Hyndman и Y. Khandakar, «Automatic Time Series Forecasting: The \texttt{forecast} Package for R», *Journal of Statistical Software*, т. 27, вып. 3, сс. 1–22, 2008, doi: [10.18637/jss.v027.i03](https://doi.org/10.18637/jss.v027.i03).
- [25] G. Ke и др., «LightGBM: A Highly Efficient Gradient Boosting Decision Tree», т. 30, 2017.
- [26] O. B. Sezer, M. U. Gudelek, и A. M. Ozbayoglu, «Financial time series forecasting with deep learning: A systematic literature review: 2005–2019», *Applied Soft Computing*, т. 90, с. 106181, 2020, doi: [10.1016/j.asoc.2020.106181](https://doi.org/10.1016/j.asoc.2020.106181).
- [27] T. Fischer и C. Krauss, «Deep learning with long short-term memory networks for financial market predictions», *European Journal of Operational Research*, т. 270, вып. 2, сс. 654–669, 2018, doi: [10.1016/j.ejor.2017.11.054](https://doi.org/10.1016/j.ejor.2017.11.054).
- [28] H. Zhou и др., «Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting», в *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, сс. 11106–11115.
- [29] A. Zeng, M. Chen, L. Zhang, и Q. Xu, «Are Transformers Effective for Time Series Forecasting?», т. 37, вып. 9, сс. 11121–11128, 2023.